



Projeto 2: Revisão do Melhor Preço

Relatório de Implementação - Estrutura de Dados

Eduardo Cardoso e Talles Souza, 2026

Sumário

1	Objetivo	2
2	Arquitetura e Passo a Passo do Sistema	2
3	Estruturação de Dados e Arquivos	3
3.1	1. O Arquivo de Lista de Compras (Input)	3
3.2	2. A Arquitetura do Arquivo de Cache (Output e Persistência)	3
4	Estruturas de Dados Desenvolvidas	4
4.1	Lista Sequencial (<code>ListaSequencial<T></code>)	4
4.2	Tabela Hash Dupla (<code>TabHash<K, V></code>)	4
5	Manual de Implantação e Execução	4
5.1	Pré-requisitos	4
5.2	Passos para Execução	4
5.3	Saída Esperada	4

1 Objetivo

Este projeto é uma evolução do buscador e comparador de preços para supermercados (Projeto 1). O objetivo principal desta segunda etapa foi resolver o problema de latência e o alto custo de processamento gerado pelas requisições web, implementando uma arquitetura de **Cache Local Persistente**.

O sistema agora é capaz de memorizar os produtos pesquisados em execuções anteriores, acessando a API dos supermercados (via requisições de texto) apenas quando é estritamente necessário e realizando requisições pontuais por *ID* apenas para garantir a atualização dos preços em tempo real.

2 Arquitetura e Passo a Passo do Sistema

Para atingir os requisitos de eficiência, o sistema executa o fluxo de processamento seguindo um rigoroso passo a passo:

1. **Inicialização e Carregamento do Cache:** Ao iniciar, a classe `Main` instancia os supermercados (Giassi, Bistek, Fort). Neste momento, a classe `CacheSupermercado` busca arquivos locais `.txt` correspondentes a cada mercado. Se existirem, os dados persistidos são carregados para as Tabelas Hash em memória (RAM), garantindo acesso $O(1)$.
2. **Leitura da Cesta Desejada:** O programa lê o arquivo de entrada do usuário (`lista-compras.txt`) e converte cada linha em um objeto `ItemLista`, que armazena os critérios de busca (termos obrigatórios e marcas).
3. **Busca Otimizada (O coração desse projeto):** Para cada `ItemLista`, o `Supermercado` verifica se o termo já existe no cache local.
 - **Caso 1: Produto não encontrado.** O sistema faz uma requisição completa na API utilizando o nome do produto. Se a API não retornar nada, o sistema salva um **Cache Negativo** para não repetir essa busca no futuro. Se encontrar produtos, realiza a paginação, salva os produtos no cache local em memória e sinaliza que o cache precisa ser atualizado no disco.
 - **Caso 2: Produto encontrado.** O sistema recupera, em $O(1)$, a lista de IDs dos produtos vinculados àquele termo. Em seguida o sistema realiza uma requisição rápida à API agrupando os IDs, atualiza os preços na memória e retorna os itens.
4. **Filtragem e Orçamento:** Com os produtos (agora com preços atualizados) em mãos, o `ItemLista` filtra apenas os que correspondem às restrições do usuário. O produto mais barato é adicionado ao `Orçamento` daquele mercado.
5. **Persistência e Ranqueamento:** Ao final, o sistema ordena os orçamentos (priorizando cestas completas e depois menores preços), exibe o resultado ao usuário e invoca o método `salvaCache()`, que grava as atualizações do dia nos arquivos `.txt` dos mercados afetados.

3 Estruturação de Dados e Arquivos

3.1 1. O Arquivo de Lista de Compras (Input)

O arquivo de configuração do usuário, `lista-compras.txt`, permite que o usuário defina múltiplos produtos e suas restrições. O formato esperado para cada linha é delimitado por ponto e vírgula (;):

Termo de Busca; Termos Obrigatórios (vírgula); Marcas Aceitáveis

Exemplo:

```
arroz; tipo 1, 5kg; camil, urbano, tio joao
leite; uht; tirol, parmalat
tapioca; ;
```

Obs: Campos vazios após o ponto e vírgula significam que não há restrições obrigatórias para aquele critério.

3.2 2. A Arquitetura do Arquivo de Cache (Output e Persistência)

Para armazenar os dados com máxima eficiência e **evitar duplicação em memória**, o sistema implementa uma **Normalização de Cache via Double Hash**. O arquivo `cache_NomeDoMercado.txt` é dividido em dois tipos de registros, identificados por prefixos:

- **Prefixo TERMO (Mapeamento Busca → IDs):** Registra a palavra pesquisada e os IDs retornados pela API. Exemplo:

```
TERMO;leite uht;101,102,103
TERMO;groselha;VAZIO
```

Obs: A tag **VAZIO** demonstra o Cache Negativo (buscas sem resultados na API).

- **Prefixo PROD (Mapeamento ID → Objeto):** Registra os dados reais (entidades) vinculados a um ID único. Exemplo:

```
PROD;101;Leite UHT Integral Tirol 1L;Tirol
;4.59;789123>true
```

Essa separação garante que, se o usuário pesquisar "leite" e "leite uht", o mesmo produto da Tirol será salvo no arquivo PROD apenas uma vez, economizando espaço em disco e garantindo complexidade $O(1)$ na atualização de preços.

4 Estruturas de Dados Desenvolvidas

4.1 Lista Sequencial (`ListaSequencial<T>`)

Estrutura central do projeto (desenvolvida em sala). Utilizada para o armazenamento das coleções, como cestas de desejos, controle de paginação de IDs e armazenamento de orçamentos. Fornece acesso indexado em tempo constante $O(1)$ e inclui a implementação do **Merge Sort** (`ordenaMescla`) para organizar os orçamentos com complexidade $O(n \log n)$.

4.2 Tabela Hash Dupla (`TabHash<K, V>`)

Também desenvolvida em sala. Responsável pelo coração da nova versão do sistema. Instanciada duplamente dentro de `CacheSupermercado`, lida com o gerenciamento assíncrono dos **TERMOS** e **PRODUTOS**. O espalhamento Hash garante consultas locais instantâneas $O(1)$ pelas palavras-chave, eliminando o gargalo de rede que existia no Projeto 1.

5 Manual de Implantação e Execução

Para garantir que seja possível reproduzir o projeto, seguem as instruções de execução:

5.1 Pré-requisitos

- Java Development Kit (JDK) 11 ou superior instalado (Recomendado Java 20+).
- Bibliotecas `org.json` e `Lombok` configuradas no `classpath` ou gerenciadas via Gradle.

5.2 Passos para Execução

1. Utilize uma IDE (como IntelliJ IDEA ou Eclipse) e importe o projeto.
2. Certifique-se de que o arquivo `lista-compras.txt` esteja criado e populado na raiz do diretório de execução.
3. Execute o método `main` da classe `Main.java`.

5.3 Saída Esperada

Se for a primeira execução (sem cache gerado), o console exibirá avisos informando a busca dos dados na API. Ao término, os orçamentos são ranqueados:

```
=====
Buscando dados na API
=====
```

```
=== RANKING DE PRECOS ===
1 Lugar - Fort: R$ 56.45
2 Lugar - Giassi: R$ 59.20
3 Lugar - Bistek (INCOMPLETA): R$ 41.10

=== DETALHES DA MELHOR CESTA ===
Supermercado: Fort (Total: R$ 56.45)
- Arroz Tipo 1 Camil 5kg (Camil): R$ 28.90
- Cafe Torrado Melitta 500g (Melitta): R$ 17.55
```

Ao realizar novamente a mesma busca, o usuário é informado que a mesma está sendo feita diretamente no cache criado anteriormente.

```
=====
Buscando dados no Cache
=====

=== RANKING DE PRECOS ===
1 Lugar - Fort: R$ 56.45
2 Lugar - Giassi: R$ 59.20
3 Lugar - Bistek (INCOMPLETA): R$ 41.10

=== DETALHES DA MELHOR CESTA ===
Supermercado: Fort (Total: R$ 56.45)
- Arroz Tipo 1 Camil 5kg (Camil): R$ 28.90
- Cafe Torrado Melitta 500g (Melitta): R$ 17.55
```